

Herança, Classes Abstratas e Polimorfismo em Java

Parte 1: Por que precisamos de herança?

O problema da duplicação de código

Imagine que uma empresa precisa cadastrar três tipos de funcionário: **Gerente**, **Vendedor** e **Estagiário**. Sem conhecer herança, você provavelmente escreveria três classes completamente separadas:

```
1 public class Gerente {
2     private String nome;
3     private String cpf;
4     private double salarioBase;
5     private double bonus;
6     // ...
7 }
8
9 public class Vendedor {
10    private String nome;           // repetido!
11    private String cpf;           // repetido!
12    private double salarioBase;   // repetido!
13    private double totalVendas;
14    // ...
15 }
16
17 public class Estagiario {
18    private String nome;           // repetido!
19    private String cpf;           // repetido!
20    private double bolsaAuxilio;
21    // ...
22 }
```

Listing 1: Tres classes com atributos privados

Repare que `nome`, `cpf` e `salarioBase` aparecem em **todas as classes**. Isso é um sinal de alerta: sempre que várias classes compartilham os mesmos atributos e comportamentos, existe uma oportunidade de organizar o código de forma melhor.

Definição: o problema que a herança resolve

Quando diversas classes têm **características em comum**, podemos extrair essas características para uma classe mais geral e fazer com que as classes específicas **reaproveitem** esse código, em vez de reescrevê-lo. A esse mecanismo damos o nome de **herança**.

Analogia: a árvore genealógica

Pense numa árvore genealógica. Um filho **herda** características dos pais — cor dos olhos, tipo sanguíneo — mas também tem as suas **próprias** características. Em Java funciona de forma parecida: uma classe pode herdar atributos e métodos de outra classe “mãe” e ainda assim adicionar as suas próprias particularidades.

Parte 2: O que é herança?

Definição: Herança

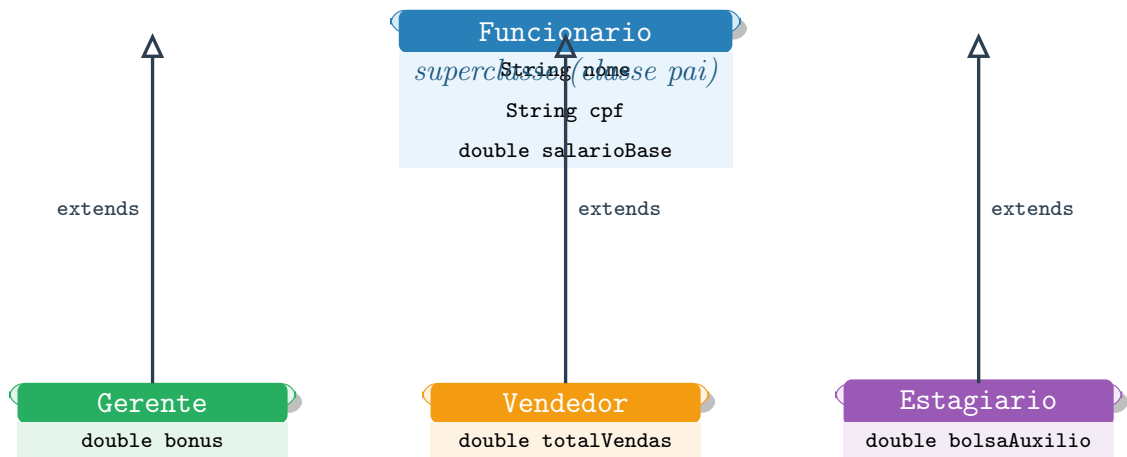
Herança é o mecanismo pelo qual uma classe (chamada **subclasse** ou **classe filha**) pode reaproveitar atributos e métodos de outra classe (chamada **superclasse** ou **classe pai**). Em Java, a herança é declarada com a palavra-chave **extends**.

```
1 public class Funcionario {  
2     private String nome;  
3     private String cpf;  
4     private double salarioBase;  
5 }  
6  
7 // Gerente ESTENDE Funcionario: herda nome, cpf e salarioBase  
8 public class Gerente extends Funcionario {  
9     private double bonus;    // atributo exclusivo do Gerente  
10 }
```

Listing 2: Sintaxe básica de herança com atributos privados

Vocabulário importante

- **Superclasse** = classe pai = classe mãe = classe base → quem **fornece** os atributos/métodos.
- **Subclasse** = classe filha = classe derivada → quem **recebe** (herda) os atributos/métodos.
- Dizemos que “Gerente **estende** Funcionario” ou “Gerente **herda de** Funcionario”.



subclasses (classes filhas) — cada uma com seu atributo exclusivo

Figure 1: As três subclasses **Gerente**, **Vendedor** e **Estagiario** herdam **nome**, **cpf** e **salarioBase** de **Funcionario**. A seta com ponta vazia aponta sempre **da subclasse para a superclasse** — é assim que a herança é representada em diagramas UML.

Verifique seu entendimento

Com essa hierarquia, os atributos **nome**, **cpf** e **salarioBase** são escritos **uma única vez**, em **Funcionario**. As três subclasses os recebem automaticamente. Isso resolve exatamente o problema de duplicação da Parte 1!

Parte 3: Tipo e subtipo — a relação “é-um”

Definição: Tipo e subtipo

Quando uma classe B estende uma classe A, dizemos que B é um **subtipo** de A, e A é o **supertipo** de B. Isso significa que **todo objeto do tipo B também é considerado um objeto do tipo A**.

Um Gerente é um Funcionario.

O teste do “é-um”

Antes de usar herança, pergunte-se: “*B é um tipo de A?*”

- *Um Gerente é um Funcionario?* Sim! → herança faz sentido.
- *Um Carro é um Motor?* Não — um carro **tem** um motor, não **é** um motor. → esse é o caso de **composição** (relação “tem-um”), que você já viu ao estudar encapsulamento e composição de objetos.

Usar herança onde a relação certa seria “tem-um” é um dos erros de projeto mais comuns em Programação Orientada a Objetos.

Relação	Pergunta-chave	Exemplo
É-um (herança)	“B é um tipo de A?”	Gerente <i>é um</i> Funcionario
Tem-um (composição)	“B possui/contém um A?”	Carro <i>tem um</i> Motor

Como consequência da relação “é-um”, uma variável do tipo da superclasse pode guardar uma referência para um objeto de qualquer subclasse. Isso é chamado de **up-casting** e será a base do **polimorfismo**, que veremos na Parte 8.

```

1 Funcionario f = new Gerente("Ana Souza", "111.222.333-44", 3000,
    1500);
2 // f é do TIPO Funcionario, mas o OBJETO real é um Gerente

```

Listing 3: Upcasting: variavel da superclasse recebe objeto da subclasse

Parte 4: O que é herdado (e o que não é)

O que a subclasse recebe automaticamente

Regra geral de herança

Uma subclasse herda todos os atributos e métodos `public` e `protected` da superclasse. Atributos e métodos `private` **existem** no objeto, mas **não podem ser acessados diretamente** pelo nome dentro da subclasse — apenas por meio de métodos públicos ou protegidos que a superclasse disponibilize.

Herança NÃO inclui construtores

Um erro comum é achar que o construtor da superclasse também é herdado. **Não é.** Cada classe deve declarar o seu próprio construtor. O que a subclasse pode fazer é **chamar** o construtor da superclasse usando `super(...)`, como veremos na Parte 5.

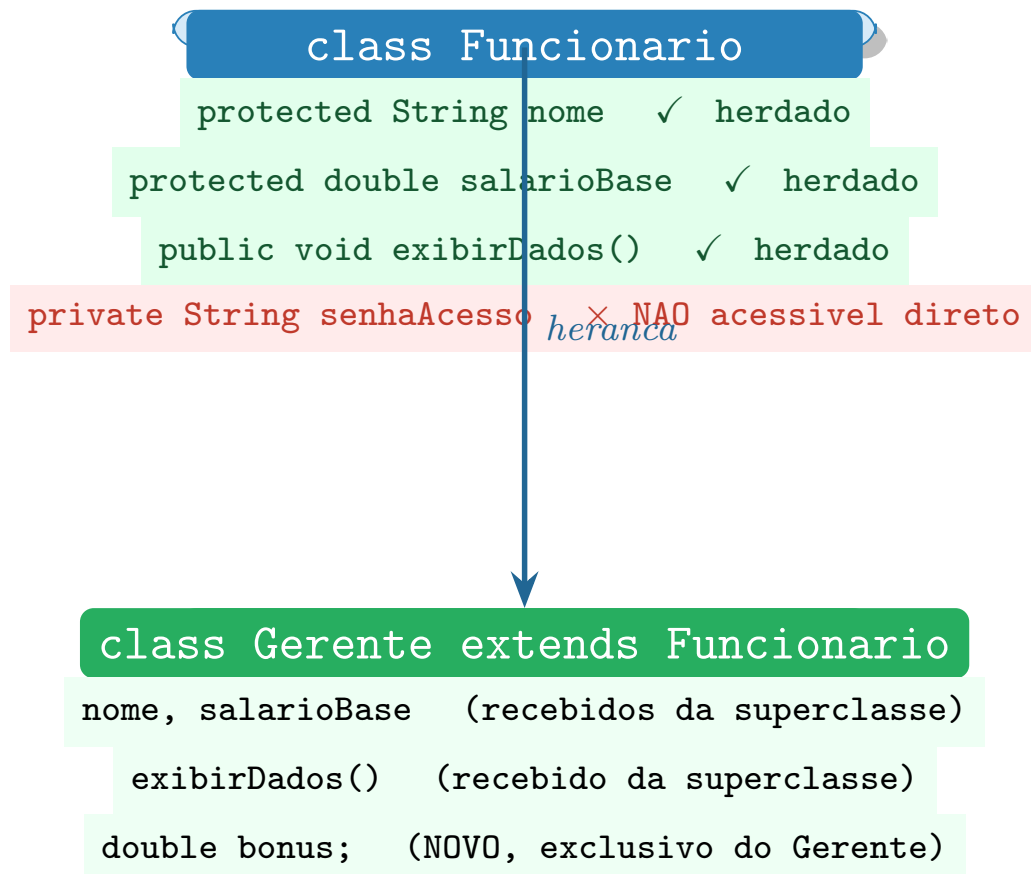


Figure 2: Membros `public` e `protected` são herdados e podem ser usados diretamente pela subclasse. Membros `private` existem no objeto, mas ficam “escondidos” — só são alcançáveis por meio de métodos que a própria superclasse exponha.

Modificadores de acesso e herança

A tabela abaixo resume quem enxerga cada membro de uma classe, considerando também o caso de herança:

Modificador	Mesma classe	Mesmo pacote	Subclasse (outro pacote)
<code>private</code>	sim	não	não
(default)	sim	sim	não
<code>protected</code>	sim	sim	sim
<code>public</code>	sim	sim	sim

Por que usar `protected` em vez de `private` na superclasse?

Quando planejamos que uma classe será estendida, é comum declarar os atributos como `protected`: assim, as subclasses conseguem acessá-los diretamente, mas o restante do programa continua sem acesso direto — o encapsulamento é mantido em relação ao mundo externo.

Parte 5: A palavra super

Definição: super

super é usado dentro de uma subclasse para se referir à **superclasse**. Tem dois usos principais: chamar o **construtor** da superclasse (`super(...)`) e chamar um **método** da superclasse (`super.metodo()`).

`super(...)`: chamando o construtor da superclasse

```
1 public class Funcionario {
2     protected String nome;
3     protected String cpf;
4     protected double salarioBase;
5
6     public Funcionario(String nome, String cpf, double salarioBase)
7     {
8         this.nome = nome;
9         this.cpf = cpf;
10        this.salarioBase = salarioBase;
11    }
12 }
13 public class Gerente extends Funcionario {
14     private double bonus;
15
16     public Gerente(String nome, String cpf, double salarioBase,
17     double bonus) {
18         super(nome, cpf, salarioBase); // chama o construtor de
19         Funcionario
20         this.bonus = bonus;           // inicializa o que e
21                                     exclusivo do Gerente
22     }
23 }
```

Listing 4: Construtor da subclasse chamando o construtor da superclasse

Regra obrigatória do `super(...)`

Se usado, `super(...)` deve ser a **primeira instrução** dentro do construtor da subclasse. Se você não escrever `super(...)` explicitamente, o Java tenta chamar automaticamente o construtor **sem parâmetros** da superclasse — e, se ele não existir, o código não compila.

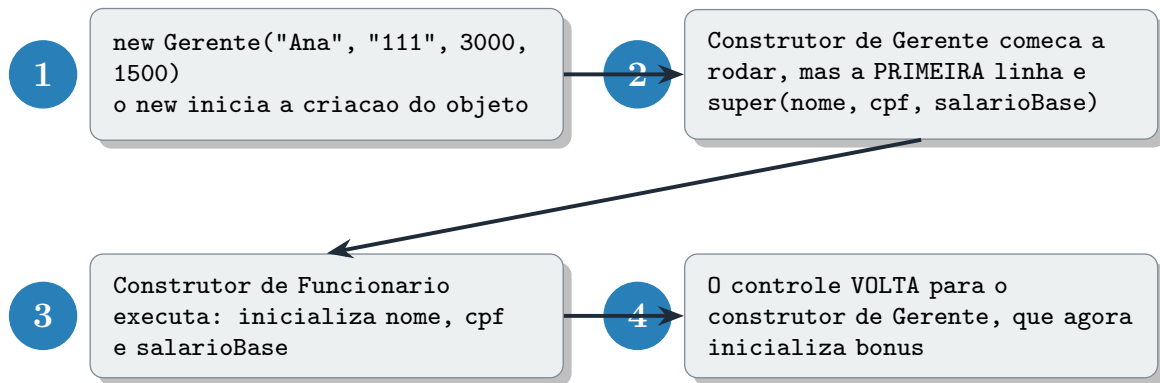


Figure 3: A superclasse sempre termina de construir a sua parte do objeto **antes** da subclasse continuar. É por isso que `super(...)` precisa ser a primeira linha do construtor.

`super.metodo()`: chamando um método da superclasse

Às vezes queremos **aproveitar** o comportamento da superclasse e apenas **acrescentar** algo a mais, em vez de reescrever tudo.

```
1 public class Gerente extends Funcionario {
2     // ...
3     @Override
4     public void exibirDados() {
5         System.out.println("--- Ficha de Gerente ---");
6         super.exibirDados();    // reaproveita o comportamento de
7         Funcionario
8     }
9 }
```

Listing 5: `super.metodo()` reaproveitando comportamento

Verifique seu entendimento

Se `Funcionario.exibirDados()` imprime nome, CPF e salário, o que a chamada `super.exibirDados()` dentro de `Gerente.exibirDados()` vai imprimir? E o que aparece **antes** disso, por causa da linha `System.out.println("-- Ficha de Gerente --")`?

Parte 6: Sobrescrita de métodos (`@Override`)

Definição: Sobrescrita (Override)

Sobrescrever um método é fornecer, na subclasse, uma **nova implementação** para um método que já existe na superclasse, mantendo **exatamente a mesma assinatura** (mesmo nome, mesmos parâmetros). A anotação `@Override` avisa o compilador — e qualquer pessoa lendo o código — que essa é a intenção.

```
1 public class Funcionario {
2     public double calcularSalario() {
3         return salarioBase;
4     }
5 }
```

```

4     }
5 }
6
7 public class Gerente extends Funcionario {
8     private double bonus;
9
10    @Override
11    public double calcularSalario() {
12        return salarioBase + bonus;    // implementacao diferente!
13    }
14 }

```

Listing 6: Sobrescrita de metodo

Por que sempre usar @Override?

Sem a anotação, um pequeno erro de digitação no nome do método (ex.: `calcularSalario` vs `calcularsalario`) cria um método **novo e diferente** em vez de sobrescrever — e o Java não avisa nada! Com `@Override`, se a assinatura não bater exatamente com um método da superclasse, o **compilador acusa erro**, evitando esse tipo de bug silencioso.

Sobrescrita (Override) x Sobrecarga (Overload)

Esses dois conceitos têm nomes parecidos em português e são frequentemente confundidos. A tabela a seguir deixa a diferença clara:

	Sobrescrita (Override)	Sobrecarga (Overload)
Onde ocorre	Entre superclasse e sub-classe	Na mesma classe
Assinatura	Deve ser idêntica (mesmo nome e parâmetros)	Mesmo nome, mas parâmetros diferentes
Objetivo	Mudar o comportamento herdado	Oferecer variações do mesmo método
Quando é decidido	Em tempo de execução (polimorfismo)	Em tempo de compilação

```

1 public class Calculadora {
2     int somar(int a, int b) {
3         return a + b;
4     }
5     double somar(double a, double b) {    // SOBRECARGA: parametros
6         return a + b;                    diferentes
7     }
8     int somar(int a, int b, int c) {      // SOBRECARGA: quantidade
9                                         diferente
10    }
11 }

```



```

9         return a + b + c;
10    }
11 }

```

Listing 7: Sobrecarga (overload): mesmo nome, parametros diferentes, mesma classe

Regras que a sobrescrita deve respeitar

- O nome do método e os tipos dos parâmetros devem ser **idênticos** aos da superclasse.
- O tipo de retorno deve ser o mesmo (ou um subtipo dele).
- A subclasse **não pode reduzir** a visibilidade do método (ex.: se era `public` na superclasse, não pode virar `private` na subclasse).

Parte 7: Classes abstratas

Um problema: faz sentido criar um “Funcionario” genérico?

Pense bem: na empresa do nosso exemplo, todo funcionário real é um **Gerente**, um **Vendedor** ou um **Estagiário**. Ninguém é “apenas um Funcionario” sem mais detalhes — essa classe existe só para **organizar o que é comum** entre as demais. Além disso, como cada tipo calcula o salário de um jeito diferente, não faz sentido nem sabemos como implementar um `calcularSalario()` genérico em `Funcionario`.

Definição: Classe abstrata

Uma **classe abstrata** é uma classe que **não pode ser instanciada diretamente** (não se pode fazer `new` nela). Ela serve apenas como **base** para outras classes. Uma classe abstrata pode ter métodos comuns já implementados e também **métodos abstratos**: métodos sem corpo, que cada subclasse concreta é **obrigada** a implementar.

Analogia: o formulário incompleto

Uma classe abstrata é como um formulário com campos que só fazem sentido depender de quem vai preenchê-lo. “Funcionario” é o modelo geral, mas o campo “como calcular o salário” só pode ser preenchido quando sabemos se é um Gerente, um Vendedor ou um Estagiário.

```

1 public abstract class Funcionario {
2
3     protected String nome;
4     protected String cpf;
5     protected double salarioBase;
6
7     public Funcionario(String nome, String cpf, double salarioBase)
8     {
9         this.nome = nome;
10        this.cpf = cpf;
11        this.salarioBase = salarioBase;

```

```

11     }
12
13     // METODO ABSTRATO: sem corpo, cada subclasse implementa do seu
14     // jeito
15     public abstract double calcularSalario();
16
17     // METODO CONCRETO: ja vem pronto, compartilhado por todas as
18     // subclasses
19     public void exhibirDados() {
20         System.out.printf("Nome: %s | CPF: %s | Salario: R$ %.2f%n",
21             nome, cpf, calcularSalario());
22     }
23 }

```

Listing 8: Classe abstrata com metodo abstrato

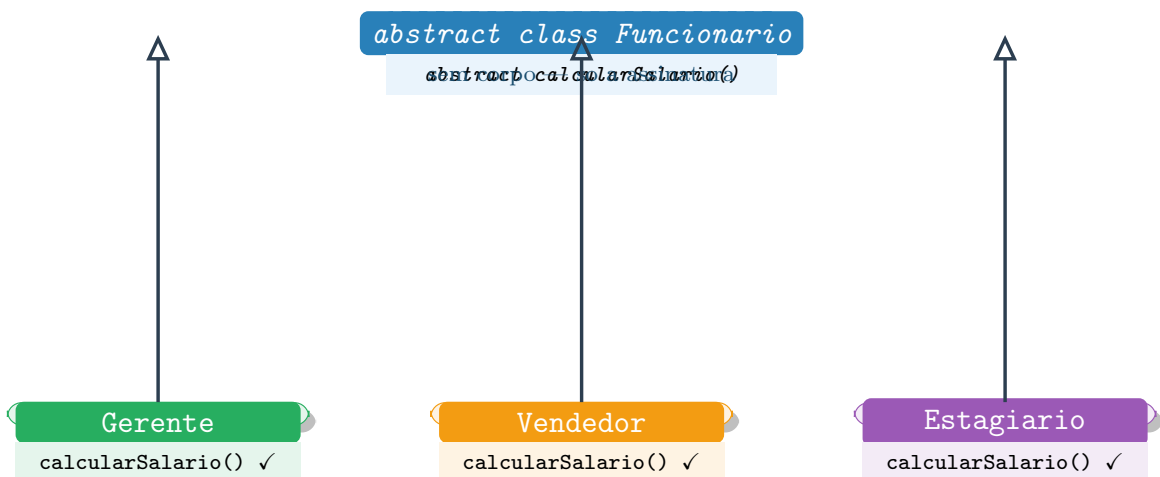
Erro: tentar instanciar uma classe abstrata

```

1 Funcionario f = new Funcionario("Ana", "111", 3000); // ERRO de
2 // compilacao!
3 // "Funcionario is abstract; cannot be instantiated"

```

Classes abstratas só podem ser usadas como **tipo de referência** (ex.: `Funcionario f = new Gerente(...)`), nunca com `new` diretamente sobre elas.



cada subclasse é OBRIGADA a implementar o método abstrato

Figure 4: `Funcionario` é abstrata (borda tracejada) e declara `calcularSalario()` apenas com a assinatura. As três subclasses concretas são obrigadas a fornecer sua própria implementação — caso contrário, o código não compila.

Regras das classes e métodos abstratos

- Uma classe com pelo menos um método abstrato **deve** ser declarada **abstract**.
- Uma classe abstrata **pode** ter métodos concretos (com corpo) além dos abstratos

— é exatamente o caso de `exibirDados()` no exemplo acima.

- Uma subclasse que **não implementa** todos os métodos abstratos herdados também precisa ser declarada `abstract`.
- Uma classe abstrata **pode** ter construtor — ele é chamado pelas subclasses através de `super(...)`, mesmo que a classe abstrata nunca seja instanciada diretamente.

Parte 8: Polimorfismo

Definição: Polimorfismo

Polimorfismo (do grego, “muitas formas”) é a capacidade de um mesmo trecho de código — uma variável do tipo da superclasse, uma chamada de método — se comportar de **formas diferentes**, dependendo do **tipo real** do objeto por trás da referência.

Analogia: o controle remoto universal

Pense num controle remoto universal com um botão “Ligar”. O botão é sempre o mesmo (a chamada do método é sempre `ligar()`), mas o que acontece de fato depende do aparelho real conectado: liga a TV, o som ou o ar-condicionado. Quem aperta o botão nem precisa saber qual aparelho está por trás — só precisa saber que existe um botão “Ligar”.

Tipo da referência x tipo do objeto real

```
1 Funcionario f = new Gerente("Ana Souza", "111.222.333-44", 3000,
    1500);
2 // tipo da REFERENCIA: Funcionario
3 // tipo do OBJETO REAL: Gerente
4
5 f.calcularSalario();
6 // o Java olha o OBJETO REAL (Gerente) para decidir qual
7 // calcularSalario() executar -- NAO o tipo da referencia!
```

Listing 9: Upcasting: a base do polimorfismo

Definição: Ligação dinâmica (dynamic binding)

Quando um método sobrescrito é chamado através de uma referência da superclasse, o Java decide, **em tempo de execução**, qual versão do método realmente rodar — olhando o **tipo real** do objeto, não o tipo declarado da variável. Esse mecanismo chama-se **ligação dinâmica**, e é ele quem torna o polimorfismo possível.

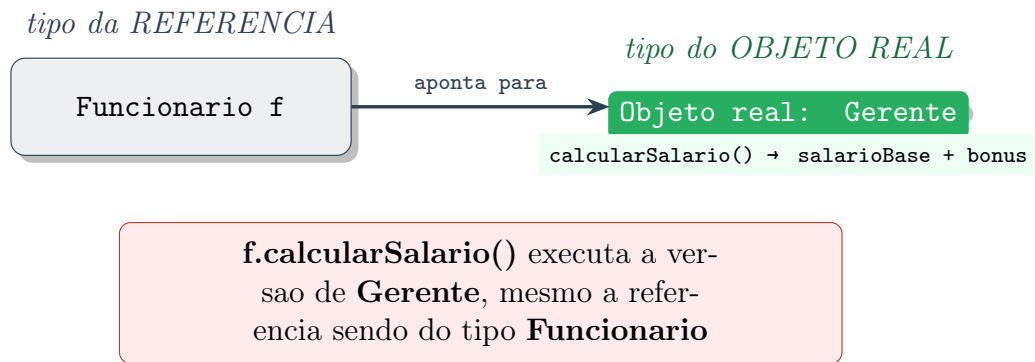


Figure 5: A variável `f` é declarada como `Funcionario`, mas aponta para um objeto `Gerente` na memória. Quando `f.calcularSalario()` é chamado, o Java executa a implementação de `Gerente` — o tipo real do objeto é quem manda.

Por que polimorfismo é útil? O mesmo código, comportamentos diferentes

A grande vantagem aparece quando guardamos **objetos de tipos diferentes** numa mesma coleção do tipo da superclasse e os tratamos de forma **uniforme**:

```
1 ArrayList<Funcionario> equipe = new ArrayList<>();
2 equipe.add(new Gerente("Ana Souza", "111", 3000, 1500));
3 equipe.add(new Vendedor("Bruno Lima", "222", 1200, 8000));
4 equipe.add(new Estagiario("Carla Dias", "333", 900));
5
6 for (Funcionario f : equipe) {
7     f.exibirDados();    // MESMA linha de código...
8 }
9 // ...mas cada chamada executa um calcularSalario() diferente!
```

Listing 10: Polimorfismo em ação com `ArrayList`

Objeto real	<code>calcularSalario()</code> executado	Por quê?
Gerente	<code>salarioBase + bonus</code>	versão de Gerente
Vendedor	<code>salarioBase + (totalVendas*0.05)</code>	versão de Vendedor
Estagiario	<code>bolsaAuxilio</code>	versão de Estagiario

A grande ideia do polimorfismo

Sem polimorfismo, o `for` acima precisaria de um `if` para cada tipo (“se for Gerente, faça X; se for Vendedor, faça Y...”). Com polimorfismo, o **mesmo laço** funciona para qualquer subtipo de `Funcionario` que existir hoje — ou que venha a existir no futuro, sem precisar alterar esse código.

Parte 9: Exemplo completo

Vamos reunir tudo o que vimos — herança, **super**, sobrescrita, classe abstrata e polimorfismo — num sistema completo de folha de pagamento.

```
1 public abstract class Funcionario {
2
3     protected String nome;
4     protected String cpf;
5     protected double salarioBase;
6
7     public Funcionario(String nome, String cpf, double salarioBase)
8     {
9         this.nome = nome;
10        this.cpf = cpf;
11        this.salarioBase = salarioBase;
12    }
13
14    public abstract double calcularSalario();
15
16    public void exibirDados() {
17        System.out.printf("Nome: %-12s | CPF: %s | Salario: R$ %.2f
18        %n",
19        nome, cpf, calcularSalario());
20    }
21 }
```

Listing 11: Funcionario.java — classe abstrata (superclasse)

```
1 public class Gerente extends Funcionario {
2
3     private double bonus;
4
5     public Gerente(String nome, String cpf, double salarioBase,
6     double bonus) {
7         super(nome, cpf, salarioBase);
8         this.bonus = bonus;
9     }
10
11    @Override
12    public double calcularSalario() {
13        return salarioBase + bonus;
14    }
15 }
```

Listing 12: Gerente.java — subclasse concreta

```
1 public class Vendedor extends Funcionario {
2
3     private double totalVendas;
4
5     public Vendedor(String nome, String cpf, double salarioBase,
6     double totalVendas) {
7         super(nome, cpf, salarioBase);
8     }
9 }
```

```

7         this.totalVendas = totalVendas;
8     }
9
10    @Override
11    public double calcularSalario() {
12        return salarioBase + (totalVendas * 0.05);
13    }
14 }

```

Listing 13: Vendedor.java — subclasse concreta

```

1 public class Estagiario extends Funcionario {
2
3     private double bolsaAuxilio;
4
5     public Estagiario(String nome, String cpf, double bolsaAuxilio)
6     {
7         super(nome, cpf, 0);
8         this.bolsaAuxilio = bolsaAuxilio;
9     }
10
11    @Override
12    public double calcularSalario() {
13        return bolsaAuxilio;
14    }
15 }

```

Listing 14: Estagiario.java — subclasse concreta

```

1 import java.util.ArrayList;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         ArrayList<Funcionario> equipe = new ArrayList<>();
7         equipe.add(new Gerente("Ana Souza", "111.111.111-11",
8         3000, 1500));
9         equipe.add(new Vendedor("Bruno Lima", "222.222.222-22",
10        1200, 8000));
11        equipe.add(new Estagiario("Carla Dias", "333.333.333-33",
12        900));
13
14        System.out.println("==== Folha de Pagamento =====");
15        for (Funcionario f : equipe) {
16            f.exibirDados();
17        }
18    }
19 }

```

Listing 15: Main.java — polimorfismo em acao

Saída esperada

```
===== Folha de Pagamento =====  
Nome: Ana Souza      | CPF: 111.111.111-11 | Salario: R$ 4500,00  
Nome: Bruno Lima    | CPF: 222.222.222-22 | Salario: R$ 1600,00  
Nome: Carla Dias     | CPF: 333.333.333-33 | Salario: R$ 900,00
```

Verifique seu entendimento

Note que Main nunca chama `calcularSalario()` diretamente — quem chama é `exibirDados()`, herdado de `Funcionario`. Mesmo assim, o salário calculado muda para cada objeto. Consegue explicar, com suas palavras, por que isso acontece?

Parte 10: Erros comuns

Erro 1: instanciar uma classe abstrata

```
1 Funcionario f = new Funcionario("Ana", "111", 3000); // ERRO de  
    compilacao!
```

Classes `abstract` nunca podem ser criadas com `new` diretamente. Crie um objeto de uma subclasse **concreta**.

Erro 2: esquecer de implementar um método abstrato

```
1 public class Diretor extends Funcionario {  
2     public Diretor(String nome, String cpf, double salarioBase)  
3     {  
4         super(nome, cpf, salarioBase);  
5         // ERRO: nao implementou calcularSalario()!  
6     }  
}
```

Toda subclasse concreta é obrigada a implementar **todos** os métodos abstratos herdados. Se `Diretor` não pretende implementar `calcularSalario()` ainda, ela também precisa ser declarada `abstract`.

Erro 3: chamar `super(...)` fora da primeira linha

```
1 public Gerente(String nome, String cpf, double salarioBase,  
    double bonus) {  
2     this.bonus = bonus;  
3     super(nome, cpf, salarioBase); // ERRO: super deve ser a 1a  
    linha!  
4 }
```

O compilador exige que `super(...)`, quando usado, seja **sempre a primeira instrução** do construtor.

Erro 4: confundir sobrescrita com sobrecarga

```
1 public class Funcionario {
2     public double calcularSalario() {
3         return salarioBase;
4     }
5 }
6 public class Gerente extends Funcionario {
7     // Assinatura diferente (parametro a mais) = SOBRECARGA, nao
7     // SOBRESCRITA!
8     public double calcularSalario(double bonusExtra) {
9         return salarioBase + bonusExtra;
10    }
11 }
```

Esse código **compila**, mas **Gerente** não sobrescreveu `calcularSalario()` — criou um método **novo**, com uma assinatura diferente. Usar `@Override` teria feito o compilador avisar sobre esse deslize.

Erro 5: esperar que a subclasse “perca” atributos privados

```
1 public class Funcionario {
2     private double salarioBase; // private!
3 }
4 public class Gerente extends Funcionario {
5     double calcularSalario() {
6         return salarioBase + bonus; // ERRO: salarioBase nao e
6         // visivel aqui
7     }
8 }
```

O atributo `salarioBase` **existe** dentro de todo objeto **Gerente**, mas por ser `private` na superclasse, não pode ser acessado diretamente pelo nome na subclasse. A solução é declará-lo `protected` (Parte 4) ou criar um método `public/protected` de acesso na superclasse.

Exercícios

Questão 1. Identificando heranca num problema. Leia o enunciado abaixo e responda:

“Uma loja de veículos cadastra Carros e Motos. Ambos têm placa, marca e ano de fabricação. Carros têm também número de portas, e motos têm cilindrada.”

- Qual seria a superclasse e quais seriam as subclasses?
- Quais atributos ficariam na superclasse? Quais são exclusivos de cada subclasse?
- Escreva a declaração da superclasse e das duas subclasses em Java, **apenas com os atributos** (sem construtores ainda).

Questão 2. Construtores e super. Usando as classes da questão anterior, escreva os construtores de `Veiculo`, `Carro` e `Moto`. Os construtores das subclasses devem usar `super(...)` para inicializar os atributos herdados.

Questão 3. Classe abstrata e polimorfismo.

Transforme `Veiculo` numa classe `abstract` com um método abstrato `double calcularImposto()`.

- a) Em `Carro`, o imposto é 2% do valor do veículo; em `Moto`, é 1%.
- b) Escreva um `Main` que crie uma coleção `ArrayList<Veiculo>` com pelo menos um carro e uma moto.
- c) Imprima o imposto de cada um usando um único laço `for` — demonstrando polimorfismo.

Questão 4. Sobrescrita x sobrecarga. Explique, com suas próprias palavras, por que o código abaixo **não** é um exemplo de sobrescrita, mesmo tendo o mesmo nome de método:

```
1 public class Animal {
2     void emitirSom() { System.out.println("..."); }
3 }
4 public class Cachorro extends Animal {
5     void emitirSom(String intensidade) { System.out.
6         println(intensidade); }
7 }
```

Em seguida, reescreva o código de forma que seja, de fato, uma sobrescrita válida (adicione `@Override`).

Questão 5. Corrija os erros. O código abaixo tem **três erros**. Identifique cada um e escreva a versão corrigida.

```
1 public abstract class Forma {
2     protected String cor;
3     public Forma(String cor) { this.cor = cor; }
4     public abstract double calcularArea();
5 }
6
7 public class Circulo extends Forma {
8     private double raio;
9     public Circulo(String cor, double raio) {
10         this.raio = raio;           // erro 1
11         super(cor);                 // erro 2
12     }
13     // erro 3: faltando a implementacao de calcularArea
14     ()
15 }
```

Questão 6. Desafio: hierarquia completa do zero. Crie uma hierarquia para um sistema de uma escola: uma classe abstrata `Pessoa` (nome, CPF) e duas subclasses, `Aluno` (matrícula, média das notas) e `Professor` (titulação, salário).

- a) Declare `Pessoa` como `abstract`, com um método abstrato `String descricao()`.
- b) Em `Aluno`, `descricao()` deve retornar algo como "Aluno matricula X, media Y".
- c) Em `Professor`, `descricao()` deve retornar algo como "Professor titulacao X, salario Y".
- d) Escreva os construtores usando `super(...)` corretamente.
- e) No `Main`, crie uma `ArrayList<Pessoa>` com alunos e professores, e imprima a `descricao()` de cada um usando um único laço `for` — demonstrando polimorfismo.

Checklist antes de entregar

- A superclasse contém apenas o que é **comum** a todas as subclasses?
- Você usou o teste do “é-um” antes de aplicar herança?
- Todo construtor de subclasse chama `super(...)` **na primeira linha**?
- Todo método sobrescrito tem a anotação `@Override`?
- Toda subclasse **concreta** implementa todos os métodos abstratos herdados?
- Você não tentou dar `new` numa classe `abstract`?
- Os atributos que as subclasses precisam acessar diretamente estão `protected`, não `private`?